

Точка входа

Введение



Для инициализации точки входа (например, поднятия TCP-сервера) не нужно использовать конструктор класса, а нужно использовать метод **Init()**.

Точкой входа называют сущность, предназначенную для генерации запросов, либо выполнения некоторой фоновой активности в рамках экземпляра сервиса БЛ.

Стандартным сценарием использования точки входа является получение запроса по одному из протоколов (HTTP, AMQP и другие), приведение его к стандартному виду, передача в ядро сервера приложений и отправка ответа. Также точка входа может исполнять определенную логику, например, рассылку логов, отслеживание изменений конфигурации приложения и прочие сервисные функции.

В Таблице 1 приведён список базовых точек входа Wasaby Framework.

Таблица 1. Базовые точки входа платформы СБИС.

Название	Название dll-модуля	Название сервисного модуля (поставляется вместе с SBIS SDK)	Описание
ConfigurationMonitor	sbis-cloud-interaction	Cloud Interaction	Поток обновления конфигурации координатора.
HeartbeatMonitor	sbis-cloud-interaction	Cloud Interaction	Поток мониторинга работоспособности пула процессов.
EntryPoint	sbis-log-sender	Log Sender	Поток ротации старых файловых логов.
LoggerConfigurator	sbis-log-sender	Log Sender	Поток конфигурирования системы отправки логов.
FcgiServerEntryPoint	sbis-fcgi-connector	Fcgi Connector	Точка входа для входящих запросов по протоколу FastCGI.
HttpServerEntryPoint	sbis-http-connector	Http Connector	Точка входа для входящих запросов по протоколу HTTP.
OFDEntryPoint	sbis-ofd-entrypoint	OFD Connector	Точка входа для обработки сообщений по протоколу ККТ-ОФД.
EntryPoint	sbis-event-subscription	EventSubscription	Точка входа для обработки входящих событий .
EntryPoint	sbis-bl-async	BL Async	Точка входа для обработки входящих асинхронных запросов .
EntryPoint	sbis-stomp-subscription	STOMP Subscription	Точка входа для обработки входящих событий по STOMP.
WebSocketEntryPoint	sbis-websocket	WebSocket	Точка входа для взаимодействия с сервисом по протоколу WebSocket и WebSocket Secure.

STOMPEntryPoint	sbis-stomp-server	STOMP Server	Точка входа для взаимодействия с сервисом по протоколу STOMP поверх WebSocket.
-----------------	-------------------	--------------	--

Пользовательская точка входа

В Wasaby Framework существует **интерфейс пользовательской точки входа**, предоставляющий возможность создания [сервисных модулей](#), способных исполнять роль источника задач для узла бизнес-логики. Задачи создаются в JSON-формате в виде POST-запроса, как будто бы они пришли снаружи сервиса. Соответственно ответ будет передан так же в JSON-формате, запакован в структуру с заголовками и передан потоком, из которого этот ответ можно прочитать.

В Wasaby Framework через данный интерфейс реализована одна точка входа, доступная в сервисном модуле **EntryPoint-Py**.

Порядок реализации пользовательской точки входа

Реализация пользовательской точки входа состоит из следующих этапов:

1. Создание нового класса, унаследованного от базового интерфейса.
2. [Описание](#) логики работы.
3. [Регистрация](#).

Требования к программному окружению

Для работы пользовательской точки входа добавьте в состав сервиса приложения сервисные модули EntryPoint и EntryPoint-Py. Они по умолчанию поставляются с составе [SBIS SDK](#).

Интерфейс пользовательской точки входа

C++

1. Подключите в состав сервиса приложения сервисный модуль EntryPoint.
2. Объявите в исходном коде новый класс, который наследуется от базового класса entryptoint::CustomEntryPoint.
3. Опишите с помощью API (см. Таблица 2) логику работы точки входа.

Таблица 2. API пользовательской точки входа на C++.

Класс	Метод	Описание
CustomEntryPoint Базовый класс для наследования в исходном коде C++.	Init()	Вызывается при активации точки входа в момент запуска сервиса. Любая инициализация должна выполняться здесь (например, создание и открытие веб-сокета).
	OnClose()	Вызывается перед остановкой сервиса. Здесь вам нужно остановить работу точки входа (например, закрыть веб-сокет).
	GetRequest() (обязательный метод)	Вызывается системой, когда она готова получить очередное задание для выполнения. Если заданий для системы в данный момент нет, можно установить ожидание в этом методе и вернуть пустую задачу. Пустые задачи не выполняются, поэтому система в этом случае сразу запросит задачу без вызова OnRequestComplete или OnRequestFail.
	OnRequestComplete()	Вызывается в случае успешного

	(обязательный метод)	выполнения задания.
	OnRequestFail() (обязательный метод)	Вызывается в случае возникновения ошибки во время выполнения задания.
	OnRequestCreated()	Вызывается в момент построения JSON-запроса и необходим только для отладки и тестов.
	MaxNumberConcurrentRequests()	Предназначен для того, чтобы сообщить системе сколько запросов может выполняться одновременно. Этот метод имеет базовую реализацию, которая разрешает выполнение одновременно <u>ЧислоРабочихПроцессов * ЧислоПотоковВРабочихПроцессах</u> . Если количество выполняемых задач будет равно максимальному, система не будет запрашивать новых задач.
	CreateRequest()	Производит создание задачи. Метод принимает название метода бизнес-логики и до 10 (для Windows) параметров к нему.
AbstractRequest Класс для описания запроса.	ExtractHeaders()	Возвращает заголовки запроса.
	ExtractId()	Возвращает идентификатор запроса.
	ExtractResponse()	Возвращает ответ.

Далее приведён пример элементарной точки входа в минимальном исполнении:

```

1 // Объявляем класс пользовательской точки входа.
2 class TestEntryPoint :public entryptoint::CustomEntryPoint
3 {
4 public:
5
6     TestEntryPoint()
7     {}
8
9     UniquePtr< rpc::AbstractRequest >::type GetRequest ( )
10    {
11        if ( !has_tasks )
12        {
13            boost::this_thread::sleep( boost::posix_time::milliseconds(
sleep_time ) );
14            return std::unique_ptr< rpc::AbstractRequest >();
15        }
16
17        auto ptr = CreateRequest( L"Object.Method", MakeParam( L"val", 10
), MakeParam( L"str_val", String( L"10" ) ) );
18        return std::move( ptr );
19    }
20
21    void OnRequestComplete ( rpc::AbstractRequest& request )
22    {
23        auto responce_header = request.GetContext()
24        if ( request.HasReply() )
25        {

```

```

26         bs::AbstractIStream response_data = request.GetReply().Data();
27         SomthingDo( response_data, response_header )
28     }
29 }
30 void OnRequestFail( rpc::AbstractRequest& request )
31 {
32     auto response_header = request.GetContext()
33     if ( request.HasReply() )
34     {
35         bs::AbstractIStream response_data = request.GetReply().Data();
36         SomthingDoOnError( response_data, response_header )
37     }
38 }
39
40 String const& Name() const
41 {
42     static String const NAME = L"TestEntryPoint";
43     return NAME;
44 }
45 };

```

Python

1. Подключите в состав сервиса приложения сервисные модули EntryPoint и EntryPoint-Py.
2. В сервисном модуле создайте новый Python-файл, в начале которого импортируйте библиотеку entryptoint.

```
1 from entryptoint import *
```

3. Объявите новый класс, который наследуется от базового класса PyEntryPointBase.
4. Опишите с помощью API (см. Таблица 3) логику работы точки входа.

Таблица 3. API пользовательской точки входа на Python.

Класс	Метод	Описание
PyEntryPointBase Базовый класс для наследования в исходном коде Python.	Init()	Вызывается при активации точки входа в момент запуска сервиса. Любая инициализация должна выполняться здесь (например, создание и открытие веб-сокета).
	OnClose()	Вызывается перед остановкой сервиса. Здесь вам нужно остановить работу точки входа (например, закрыть веб-сокет).
	GetRequest() (обязательный метод)	Вызывается системой, когда она готова получить очередное задание для выполнения. Если заданий для системы в данный момент нет, можно установить ожидание в этом методе и вернуть пустую задачу. Пустые задачи не выполняются, поэтому система в этом случае сразу запросит задачу без вызова OnRequestComplete или OnRequestFail.
	OnRequestComplete() (обязательный метод)	Вызывается в случае успешного выполнения задания.

	OnRequestFail() (обязательный метод)	Вызывается в случае возникновения ошибки во время выполнения задания.
	OnRequestCreated()	Вызывается в момент построения JSON-запроса и необходим только для отладки и тестов.
	MaxNumberConcurrentRequests	Предназначен для того, чтобы сообщить системе сколько запросов может выполняться одновременно. Этот метод имеет базовую реализацию, которая разрешает выполнение одновременно <u>ЧислоРабочихПроцессов * ЧислоПотоковВРабочихПроцессах</u> . Если количество выполняемых задач будет равно максимальному, система не будет запрашивать новых задач.
PyRequest Класс для описания запроса.	SetRequest()	Устанавливает запрос с параметрами в формате: 1 SetRequest("Метод", param1 = value1, 2 param2 = value2)
	GetParams()	Возвращает *dict* параметров запроса в формате: 1 {"param1": value1, 2 "param2": value2}
	GetMethodName()	Возвращает метод, который будет вызван.
	SetHeaders()	Устанавливает заголовки запроса.
	GetHeaders()	Возвращает dict заголовков запроса в формате: 1 { "Header1": "value1", 2 "Header2": "value2" }
	Id()	Возвращает идентификатор запроса.
PyResponse Класс с содержимым ответа. Для получения данных реализует стандартный интерфейс потока.	GetHeaders()	Возвращает *dict* заголовков ответа в формате: 1 { "Header1": "value1", 2 "Header2": "value2" }
	Id()	Возвращает идентификатор запроса, которому соответствует ответ. 1 PyRequest.Id() == PyResponse.Id()

Далее приведён пример элементарной точки входа в минимальном исполнении, которая описана в файле `my_test_entry_point.py`:

```
1  # Импортируем API для работы с точками входа.
2  from entrypoint import *
3
4  # Объявляем класс пользовательской точки входа.
5  class TestPyEntryPoint(PyEntryPointBase):
6      def __init__( self ):
7          pass
8
9      # Обрабатываем запрос.
10     def GetRequest( self ):
11         if has_tasks == False:
12             sleep(1)
13             return None
14         req = PyRequest()
15         req.SetRequest( "Object.Method", val = 10, str_val = "10" )
16         req.SetHeaders( { "Header1" : "value1", "Header2" : "value2" } )
17         method = req.GetMethodName();
18         params = req.GetParams();
19         headers = req.GetHeaders();
20         id = req.Id();
21         return req
22
23     # Обрабатываем успешное завершение запроса.
24     def OnRequestComplete( self, r ):
25         response_str = r.read().decode( 'utf-8' )
26         headers = r.GetHeaders();
27         id = r.Id(); # Равен id соответствующего запроса.
28
29     # Обрабатываем завершение запроса при ошибке.
30     def OnRequestFail( self, r ):
31         response_str = r.read().decode( 'utf-8' )
32         headers = r.GetHeaders();
33         id = r.Id(); # Равен id соответствующего запроса.
```

Регистрация точки входа

C++

Для регистрации точки входа используйте макрос `REGISTER_ENTRY_POINT`:

```
1  REGISTER_ENTRY_POINT( TestEntryPoint )
```

Выполните эту инструкцию в том же CPP-файле, где описана пользовательская точка входа.

Python

1. Найдите файл `on_event.py` в сервисном модуле, в котором находится python-файл с объявленной пользовательской точкой входа. Если файла нет, создайте его и поместите в корень директории такого модуля.
2. В файле `on_event.py` выполните следующие действия:
 - импортируйте библиотеку `entrypoint`, чтобы получить доступ к API регистрации точки входа;

- импортируйте класс, в котором объявлена пользовательская точка входа;
- в методе OnEndAllLoadModules() (см. [Методы, вызываемые при загрузке сервисного модуля](#)) вызовите функцию RegisterEntryPoint() для регистрации точки входа.

```
1 # Импортируем класс с описанием пользовательской точки входа.
2 from my_test_entry_point import TestPyEntryPoint
3
4 # Импортируем API для работы с точками входа.
5 from entrypoint import *
6
7 # Регистрация точки входа выполняется после загрузки модулей.
8 def OnEndAllLoadModules(answer):
9     try:
10         pass
11
12         # Регистрация.
13         RegisterEntryPoint( TestPyEntryPoint() )
14     except Exception:
15         pass
16     return True
```